

Exercício (Novo): API de Tarefas — "Meu To-Do"

Turma: LionsDev

Tópicos: Boilerplate, rotas protegidas com JWT, dono do recurso via token, `Boolean` e `enum` no Mongoose, ação customizada (`PATCH /:id/concluir`), Query Params e status codes.

Nível: porta de entrada. Comece por este antes de Finanças e da Lions Bet.

1. Contexto

Você vai construir uma **lista de tarefas pessoal**. Cada usuário faz login e gerencia **apenas as próprias tarefas**: cria, marca como concluída, edita e apaga. É o "Hello World" das APIs com login — simples, mas com todas as peças do boilerplate.

2. Ponto de Partida: o Boilerplate

Partimos do **boilerplate LionsDev**: <https://github.com/nicolassmotta/boilerplate-lions-dev.git>

1. Clone, `npm install`, crie o `.env` (`MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `BCRYPT_SALT_ROUNDS`) e suba o servidor.
2. Já estão prontos: camada de `Usuario`, cadastro/login com `bcrypt/JWT`, middleware `autenticar` (preenche `req.usuario = { id, email }`), `criarErro` e o middleware de erro.

Antes de começar, pegue seu token e envie `Authorization: Bearer SEU_TOKEN` em todas as rotas novas.

3. Regras de Ouro

1. **Toda rota é protegida.** `router.use(autenticar)` no topo do arquivo de rotas.
2. **O dono vem do token** (`req.usuario.id`), **nunca do body**.
3. **Toda consulta filtra pelo dono** (`{ _id: idTarefa, usuario: idDoUsuario }`). Se não achar → `404`.
4. **Listagem nunca dá 404:** sem tarefas, devolva array vazio.

4. Arquivos que você vai criar

Camada	Arquivo
Model	<code>src/models/tarefa.model.js</code>
Repository	<code>src/repositories/tarefa.repository.js</code>
Service	<code>src/services/tarefa.service.js</code>
Controller	<code>src/controllers/tarefa.controller.js</code>
Routes	<code>src/routes/tarefa.routes.js</code>

5. Etapa 1 — Model Tarefa

- `titulo` : `String` , obrigatório, `trim` , `minlength: [2, "0 título deve ter pelo menos 2 caracteres."]` .
- `descricao` : `String` , opcional, `trim` .
- `prioridade` : `String` , `enum: ["baixa", "media", "alta"]` , `default: "media"` .
- `concluida` : `Boolean` , `default: false` .
- `dataLimite` : `String` , opcional (ex.: `"2026-06-30"`).
- `usuario` : `ObjectId` , `ref: "Usuario"` , obrigatório.
- `timestamps: true` .

Critérios de aceite: `titulo` e `usuario` são obrigatórios; `prioridade` só aceita os três valores; tarefa nova nasce `concluida: false` .

Conceito novo — `enum` , `default` e o campo de dono (`ObjectId` + `ref`)

Três coisas no Schema podem ser novidade:

- `enum` trava o campo a uma lista fechada. Qualquer valor fora dela vira erro de validação (responde `400`).
- `default` é o valor inicial quando o body não manda o campo (ex.: `concluida` nasce `false` sozinha).
- `usuario` guarda o **dono** da tarefa. É uma *referência* ao `_id` de um `Usuario` , por isso o tipo é `ObjectId` com `ref: "Usuario"` .

Sintaxe desses campos (o resto do Schema você monta como já sabe):

```
import mongoose from "mongoose";
// ...dentro do new mongoose.Schema({ ... })
prioridade: { type: String, enum: ["baixa", "media", "alta"], default:
"media" },
concluida: { type: Boolean, default: false },
usuario: { type: mongoose.Schema.Types.ObjectId, ref: "Usuario",
required: true },
```

6. Etapa 2 — Repository

- `criar(dados) → Tarefa.create(dados)` .
- `listarPorUsuario(idUsuario, filtro = {})` → `Tarefa.find({ usuario: idUsuario, ...filtro }).sort({ createdAt: -1 })` .
- `buscarPorIdDoDono(idTarefa, idUsuario)` → `Tarefa.findOne({ _id: idTarefa, usuario: idUsuario })` .
- `atualizarPorIdDoDono(idTarefa, idUsuario, dados)` → `findOneAndUpdate(filtro, dados, { new: true, runValidators: true })` .
- `deletarPorIdDoDono(idTarefa, idUsuario)` → `findOneAndDelete(filtro)` .
- Exporte tudo no objeto `TarefaRepository` .

Repare no `...filtro` : ele permite adicionar `{ concluida: true }` na listagem sem quebrar o filtro por dono. Veremos isso no bônus.

Conceito novo — por que filtrar pelo dono já na consulta (`{ _id, usuario }`)

A segurança aqui **não** usa `if (tarefa.usuario === req.usuario.id)` . Em vez disso, você já pede ao banco só o que é seu: o filtro leva **dois** campos juntos — o id da tarefa **e** o id do dono (que veio do token, `req.usuario.id`).

```
// repository – buscar/atualizar/remover sempre levam o dono no filtro
Tarefa.findOne({ _id: idTarefa, usuario: idUsuario });
Tarefa.findOneAndUpdate({ _id: idTarefa, usuario: idUsuario }, dados, {
new: true, runValidators: true });
Tarefa.findOneAndDelete({ _id: idTarefa, usuario: idUsuario });
```

Se a tarefa não existe **ou** é de outra pessoa, o banco devolve **null** e o service responde **404**. "Não existe" e "não é sua" viram a mesma resposta de propósito — não vaza informação.

7. Etapa 3 — Service

- `registrar(idDoUsuario, dados)` : monta o objeto com `usuario: idDoUsuario` e cria.
- `listarMinhas(idDoUsuario)` : retorna a lista do usuário.
- `buscarMinha(idDoUsuario, idTarefa)` : `buscarPorIdDoDono(...)` ; se `null` → `criarErro("Tarefa não encontrada.", 404)` .
- `atualizarMinha(idDoUsuario, idTarefa, dados)` : `atualizarPorIdDoDono(...)` ; se `null` → `404` .
- `concluirMinha(idDoUsuario, idTarefa)` : atualiza a tarefa setando `{ concluida: true }` ; se `null` → `404` .
- `removerMinha(idDoUsuario, idTarefa)` : `deletarPorIdDoDono(...)` ; se `null` → `404` ; se ok → `{ message: "Tarefa removida com sucesso." }` .

8. Etapa 4 — Controller e Etapa 5 — Routes

Crie o controller (`try/catch` + `next(error)`) e as rotas. No `src/routes/tarefa.routes.js` , com `router.use(authenticar)` no topo:

- `POST /` → registrar (`201`)
- `GET /` → listarMinhas (`200` com `{ tarefas }`)
- `GET /:id` → buscarMinha (`200`)
- `PATCH /:id` → atualizarMinha (`200`)
- `PATCH /:id/concluir` → concluirMinha (`200`)
- `DELETE /:id` → removerMinha (`200`)

No `src/app.js` , antes do middleware 404:

```
import tarefaRoutes from "../routes/tarefa.routes.js";
app.use("/api/tarefas", tarefaRoutes);
```

Declare `PATCH /:id/concluir` antes de `PATCH /:id` não é obrigatório (são caminhos diferentes), mas mantenha as rotas organizadas e legíveis.

9. Rotas finais

Método	Rota	Proteção	Descrição
POST	<code>/api/tarefas</code>	JWT	Criar tarefa (dono = você)
GET	<code>/api/tarefas</code>	JWT	Listar minhas tarefas
GET	<code>/api/tarefas/:id</code>	JWT	Ver uma tarefa minha
PATCH	<code>/api/tarefas/:id</code>	JWT	Editar uma tarefa minha
PATCH	<code>/api/tarefas/:id/concluir</code>	JWT	Marcar tarefa como concluída
DELETE	<code>/api/tarefas/:id</code>	JWT	Remover uma tarefa minha

10. Fluxo de Testes (use Postman)

1. Cadastro/login → copie o `token`.
2. `POST /api/tarefas` com `{ "titulo": "Estudar JWT", "prioridade": "alta" }` → `201`, `concluida: false`.
3. `GET /api/tarefas` → lista só as suas.
4. `PATCH /api/tarefas/:id/concluir` → `200`, `concluida: true`.
5. `PATCH /api/tarefas/:id` com `{ "prioridade": "baixa" }` → `200`.
6. `DELETE /api/tarefas/:id` → `200`; repita → `404`.
7. Com um **segundo usuário**, tente ver uma tarefa do primeiro → `404`.
8. Qualquer rota **sem token** → `401`.

11. Desafios Bônus

1. `GET /api/tarefas?concluida=true` — leia `req.query.concluida` no controller e repasse ao service para filtrar **entre as suas** (lembre: query chega como texto `"true"` / `"false"`).
2. `GET /api/tarefas?prioridade=alta` — filtra suas tarefas por prioridade.
3. `GET /api/tarefas/resumo` — em JavaScript, conte quantas tarefas suas estão concluídas e quantas estão pendentes.

Como ler um Query Param (`?concluida=true`): no controller, `req.query.concluida` chega sempre como **texto** (`"true"`), nunca como booleano. Converta antes de mandar pro service:

```
// controller
const filtro = {};
if (req.query.concluida !== undefined) {
  filtro.concluida = req.query.concluida === "true"; // vira true/false
  de verdade
}
const tarefas = await TarefaService.listarMinhas(req.usuario.id,
filtro);
```

LionsDev • Professor Nicolas Cardoso Motta

Exercício novo de API com Boilerplate (Auth + camadas) - Módulo 09